

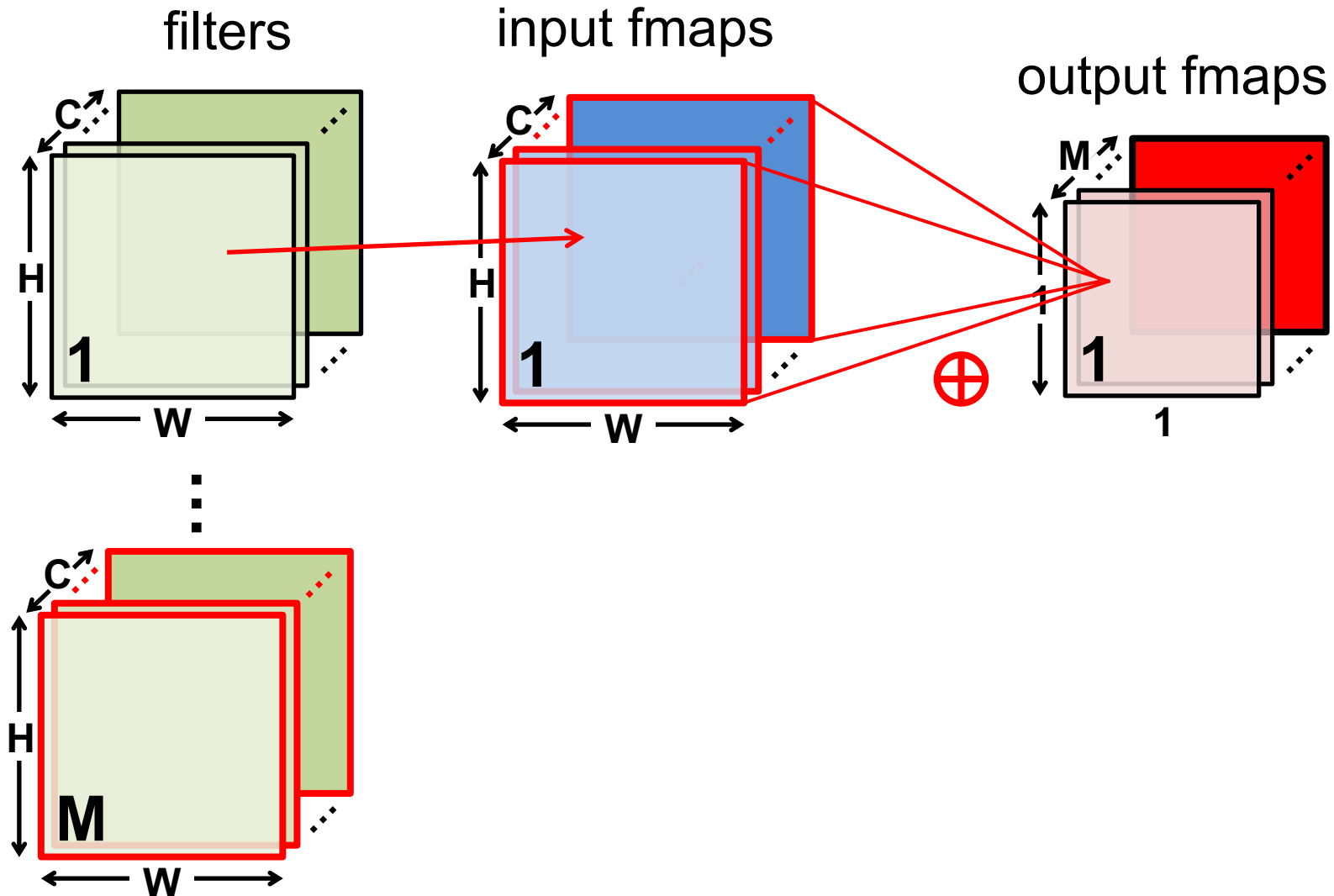
Kernel Computation

ISCA Tutorial (2019)

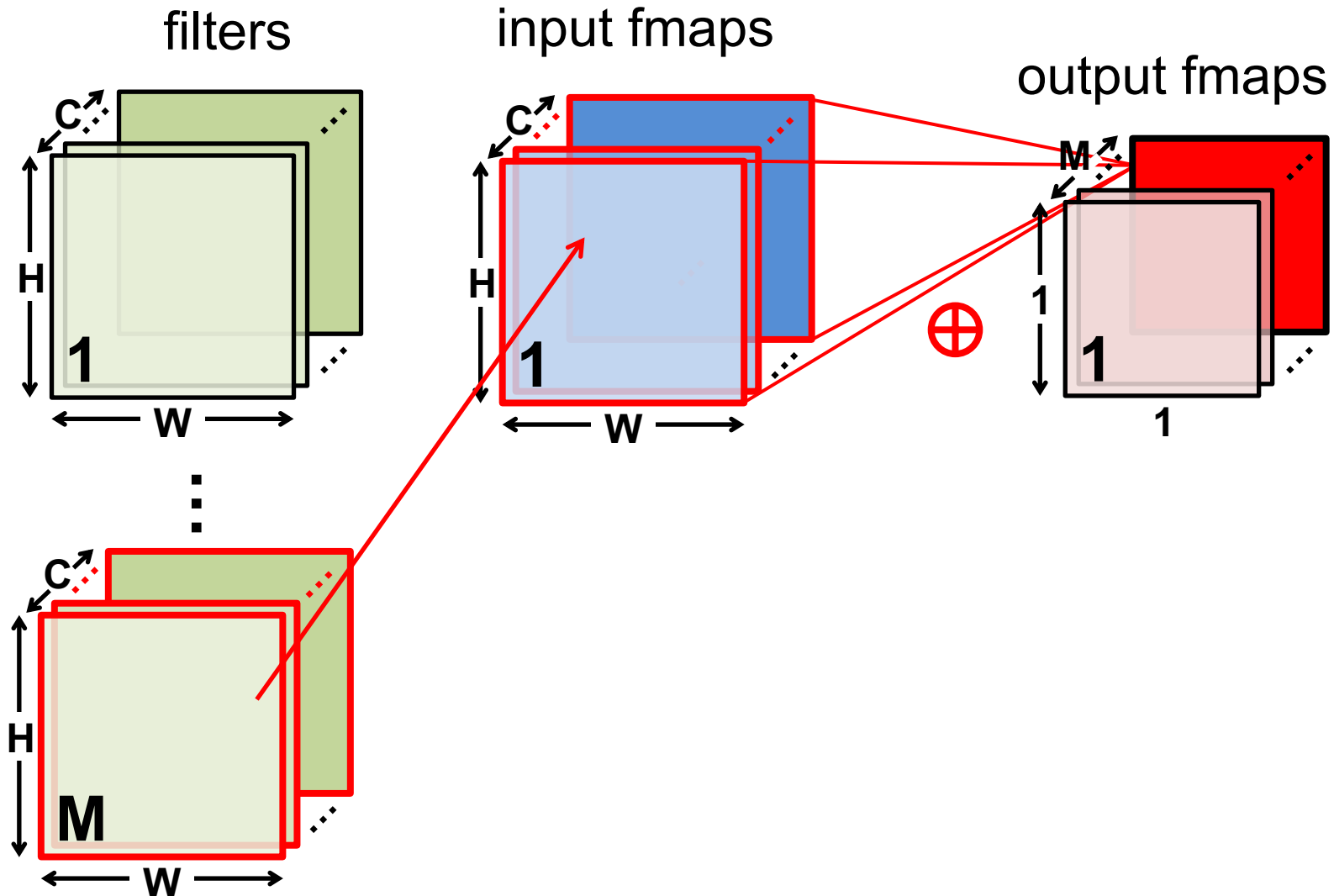
Website: <http://eyeriss.mit.edu/tutorial.html>

Joel Emer, Vivienne Sze, Yu-Hsin Chen

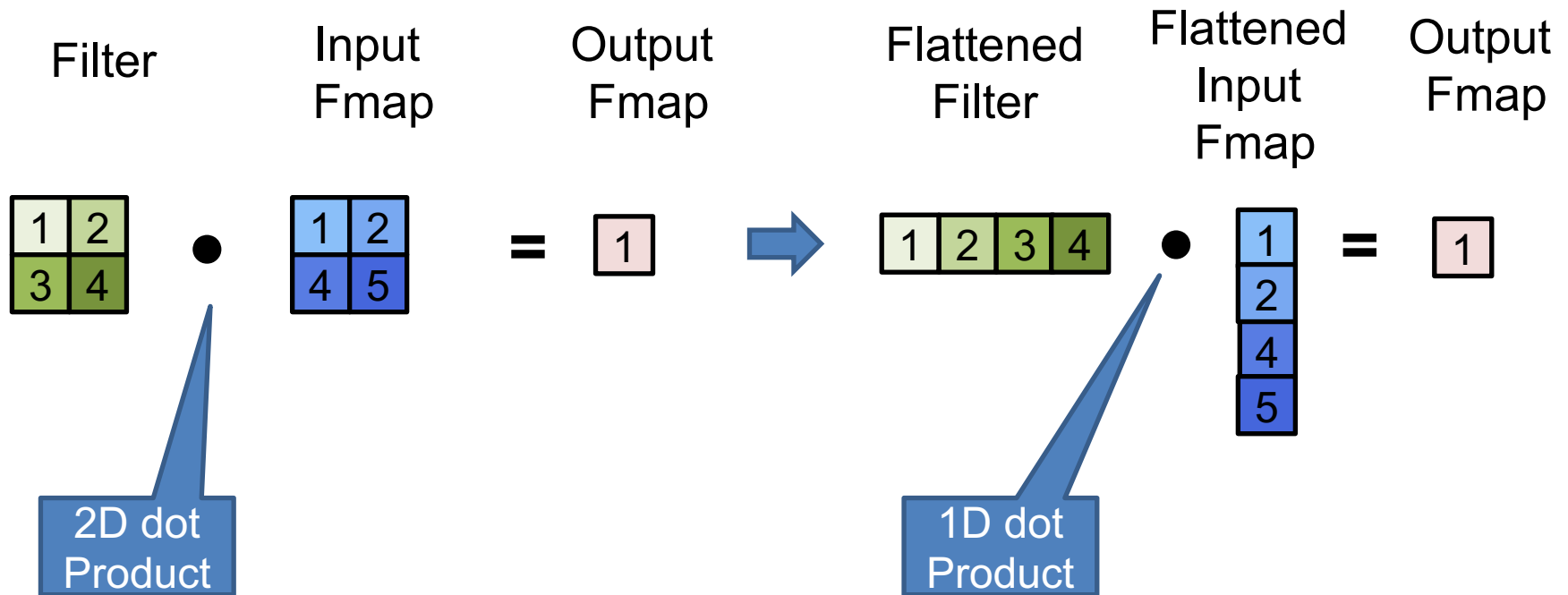
Fully-Connected (FC) Layer



Fully-Connected (FC) Layer

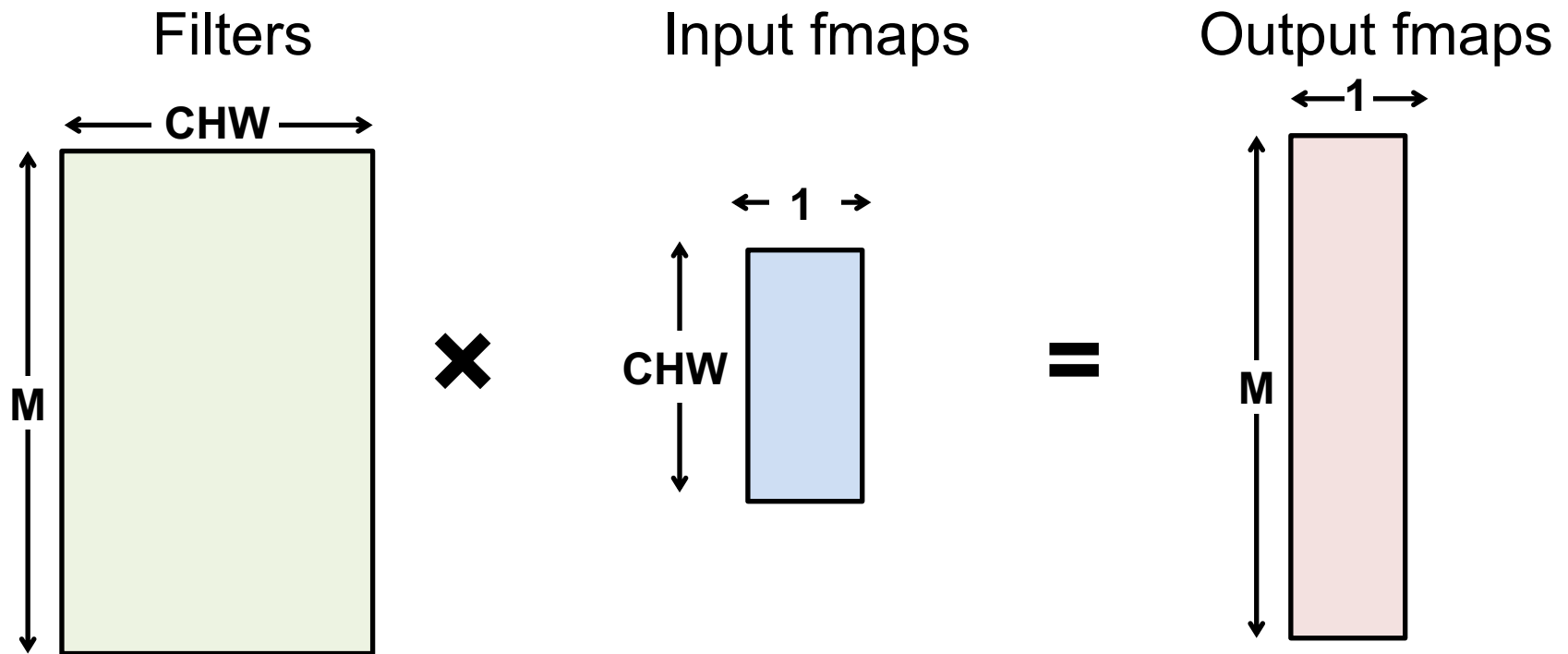


Flattened 2D Dot Product



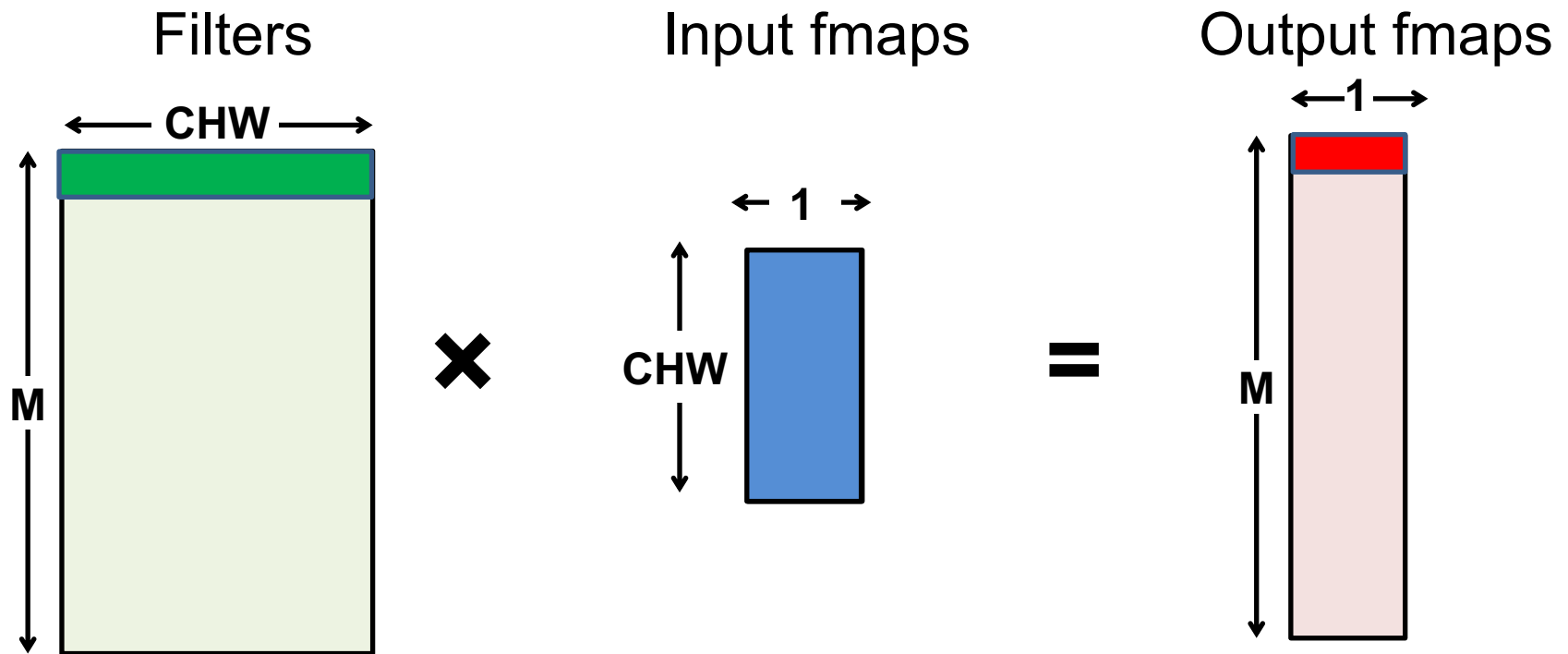
Fully-Connected (FC) Layer

- Matrix–Vector Multiply:
 - Multiply all inputs in all channels by a weight and sum



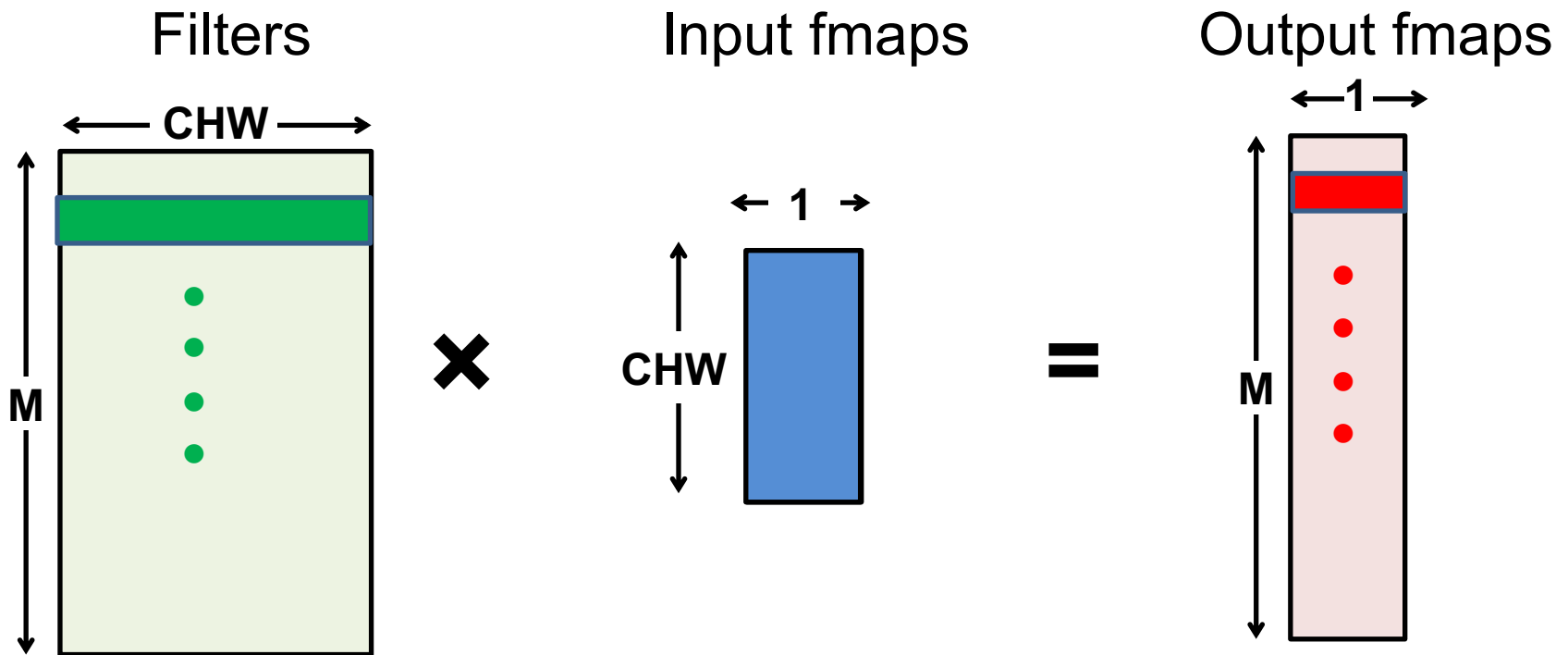
Fully-Connected (FC) Layer

- Matrix–Vector Multiply:
 - Multiply all inputs in all channels by a weight and sum

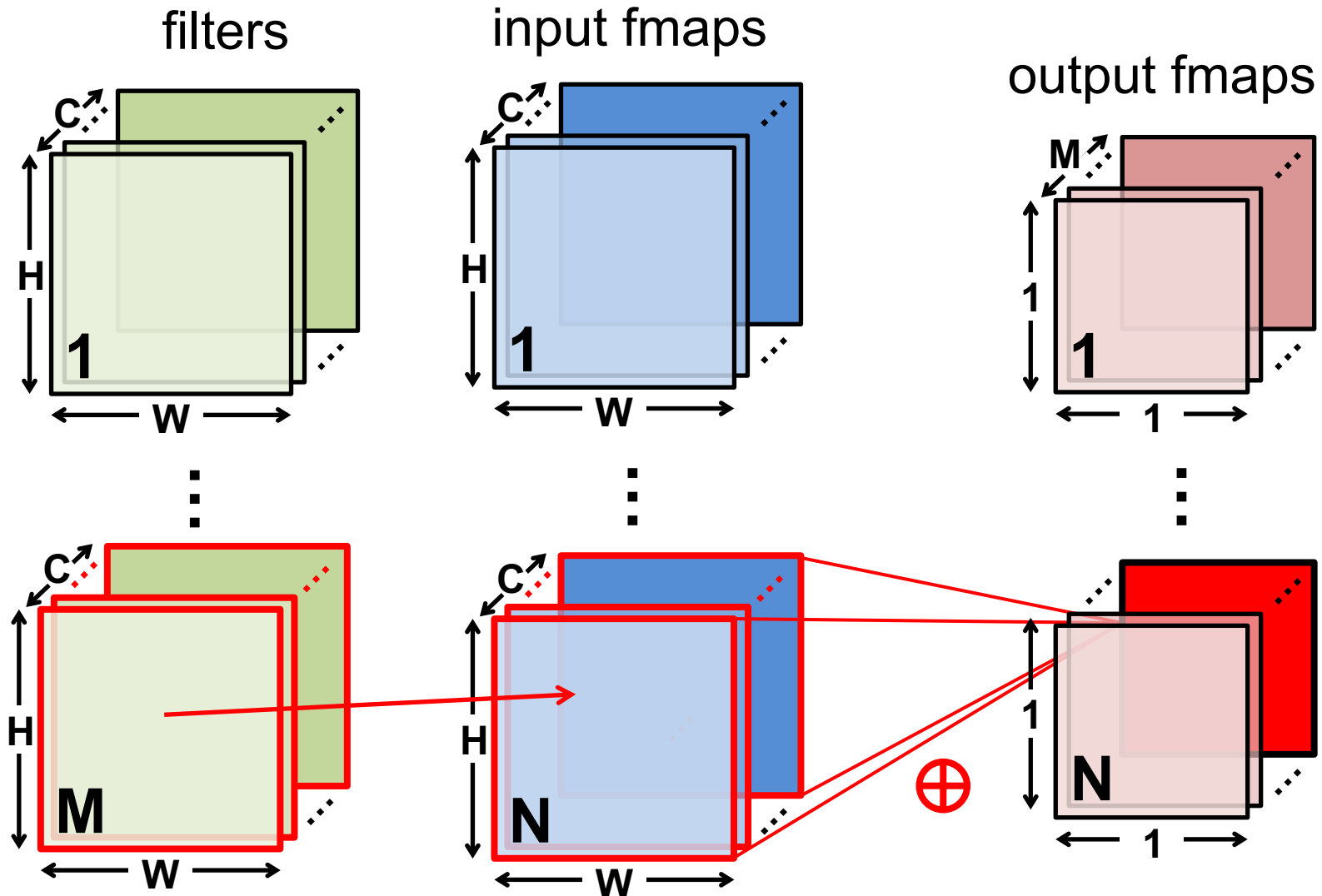


Fully-Connected (FC) Layer

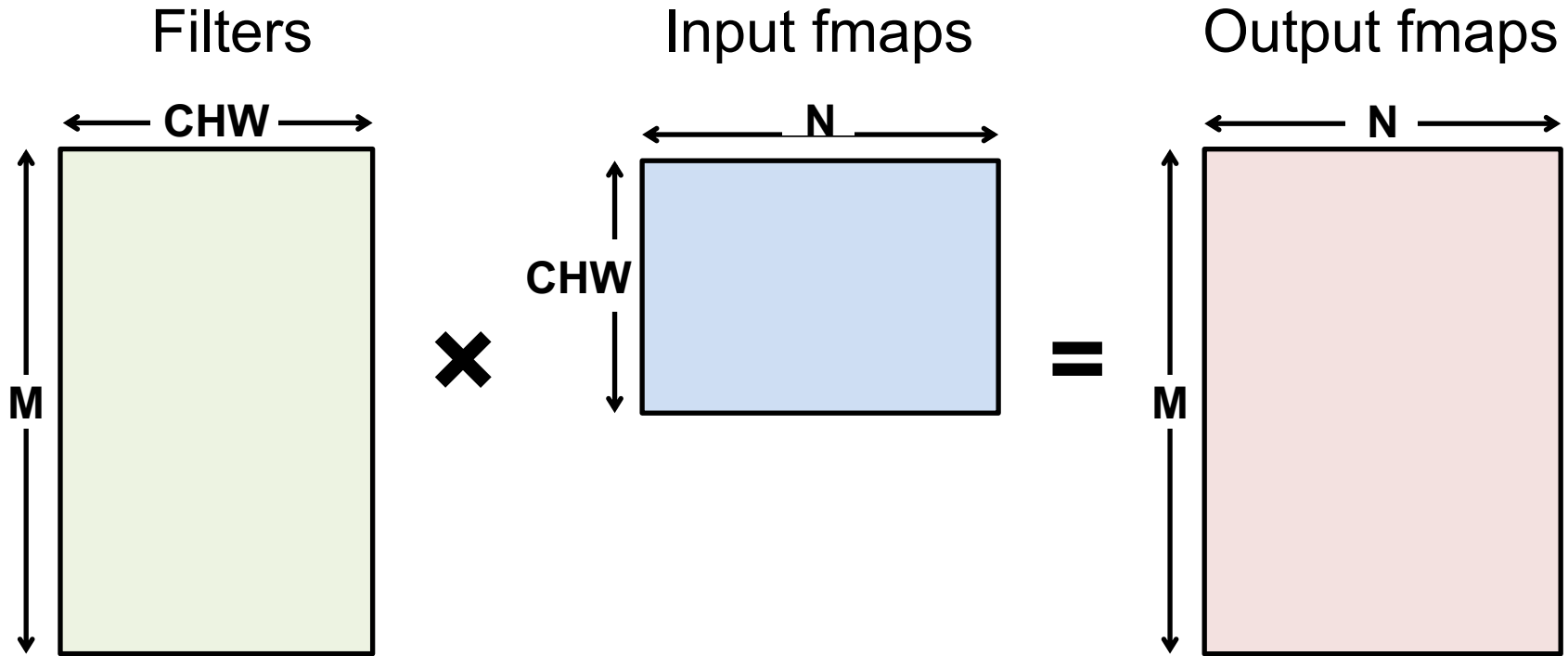
- Matrix–Vector Multiply:
 - Multiply all inputs in all channels by a weight and sum



Fully-Connected (FC) Layer

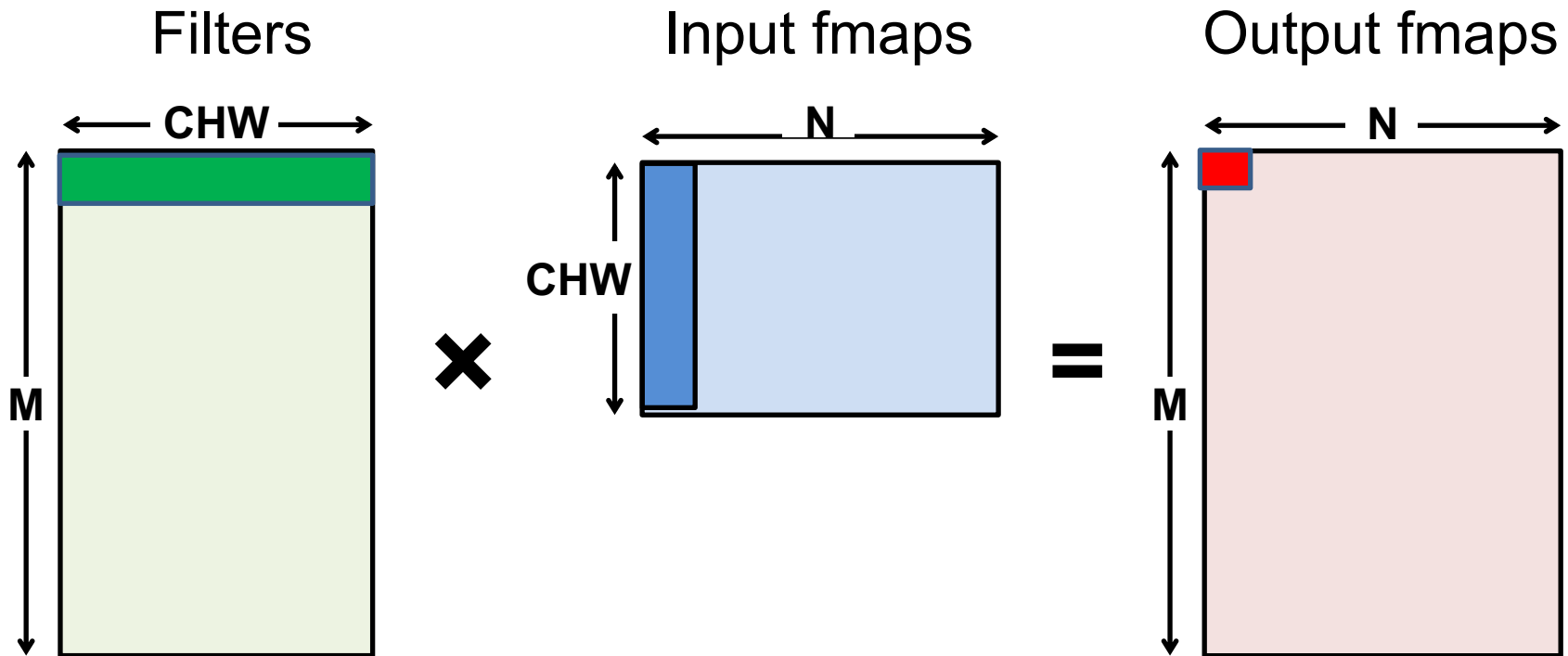


Flattened Fully-Connected Layer



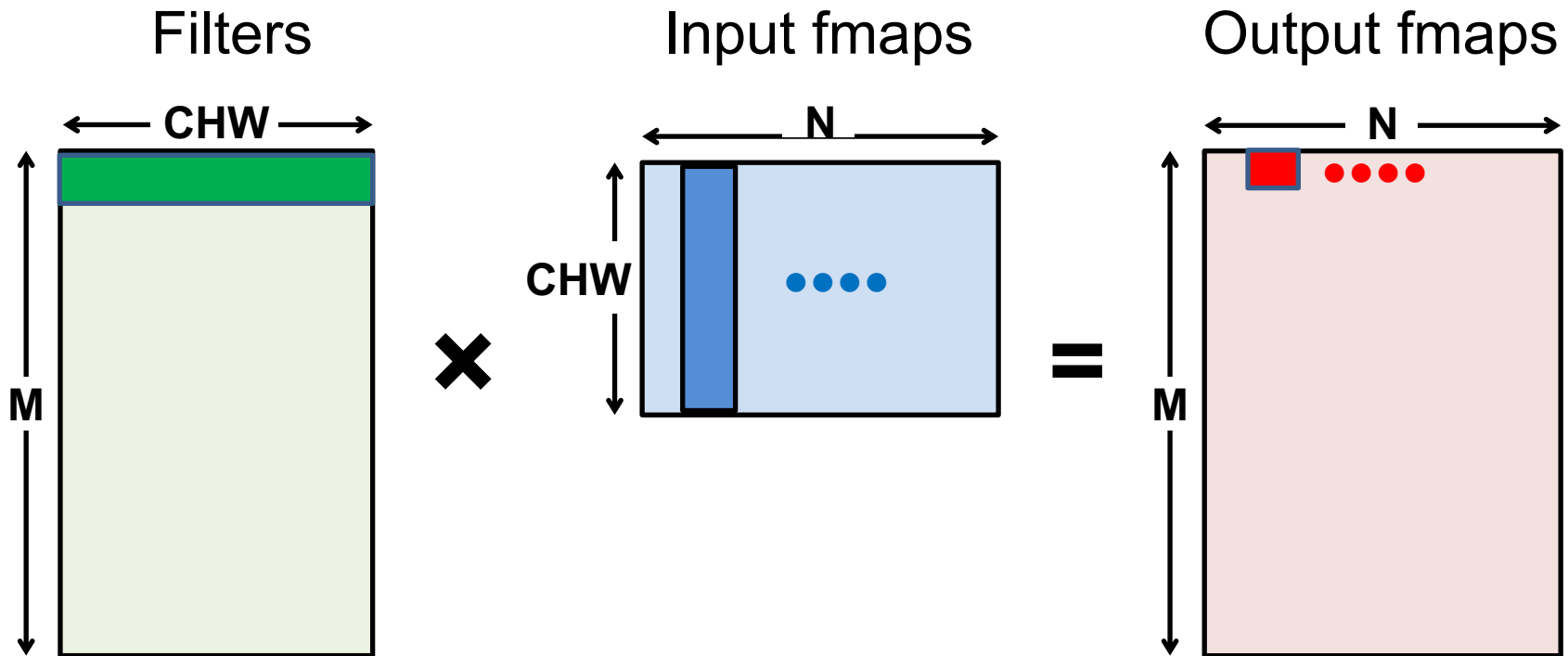
- After flattening, having a batch size of N turns the matrix-vector operation into a matrix-matrix multiply

Fully-Connected (FC) Layer



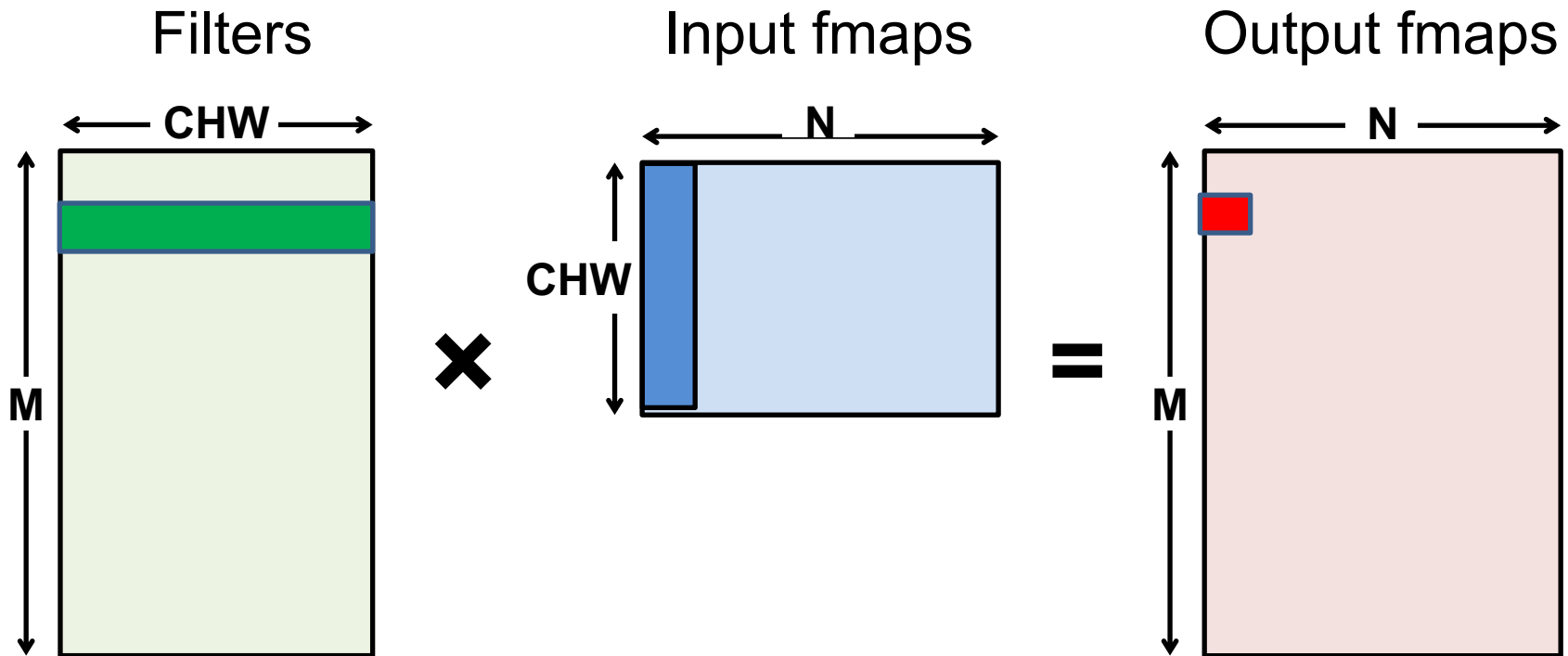
- After flattening, having a batch size of N turns the matrix-vector operation into a matrix-matrix multiply

Fully-Connected (FC) Layer



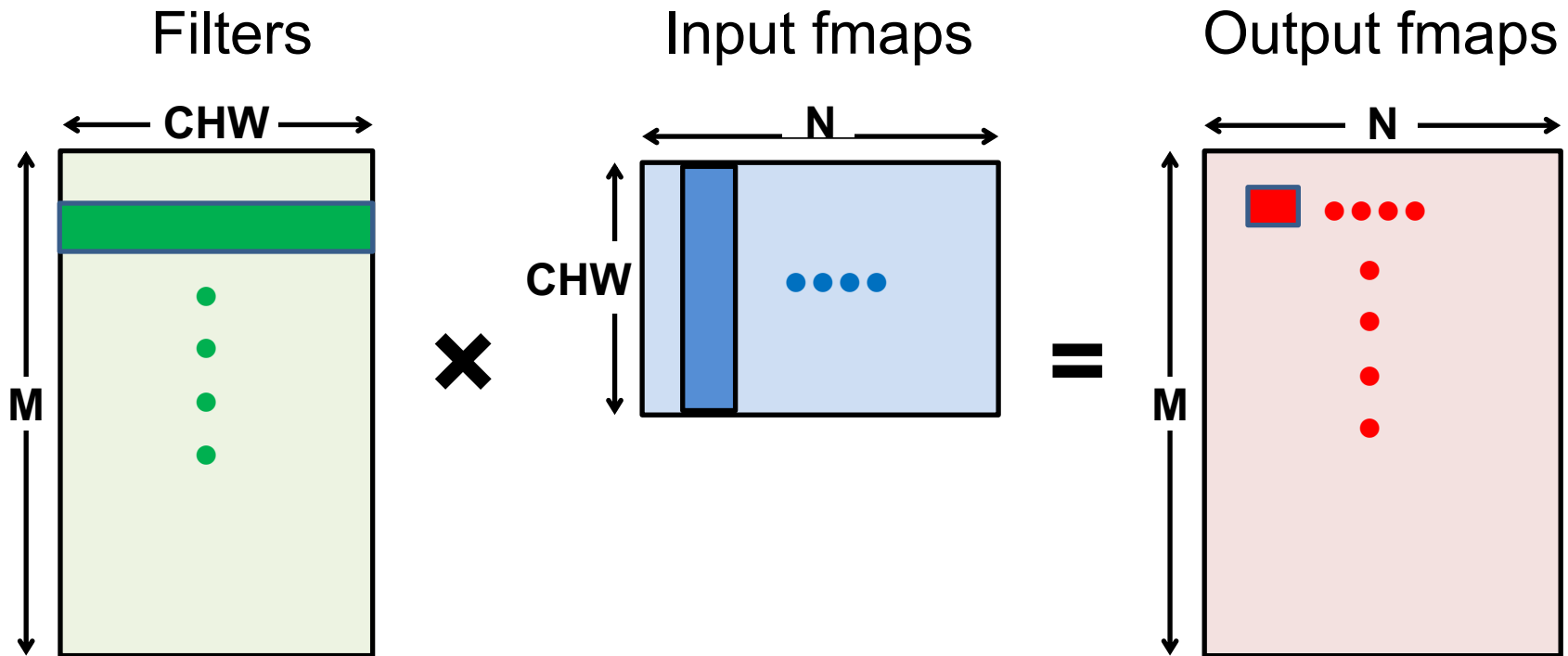
- After flattening, having a batch size of N turns the matrix-vector operation into a matrix-matrix multiply

Fully-Connected (FC) Layer



- After flattening, having a batch size of N turns the matrix-vector operation into a matrix-matrix multiply

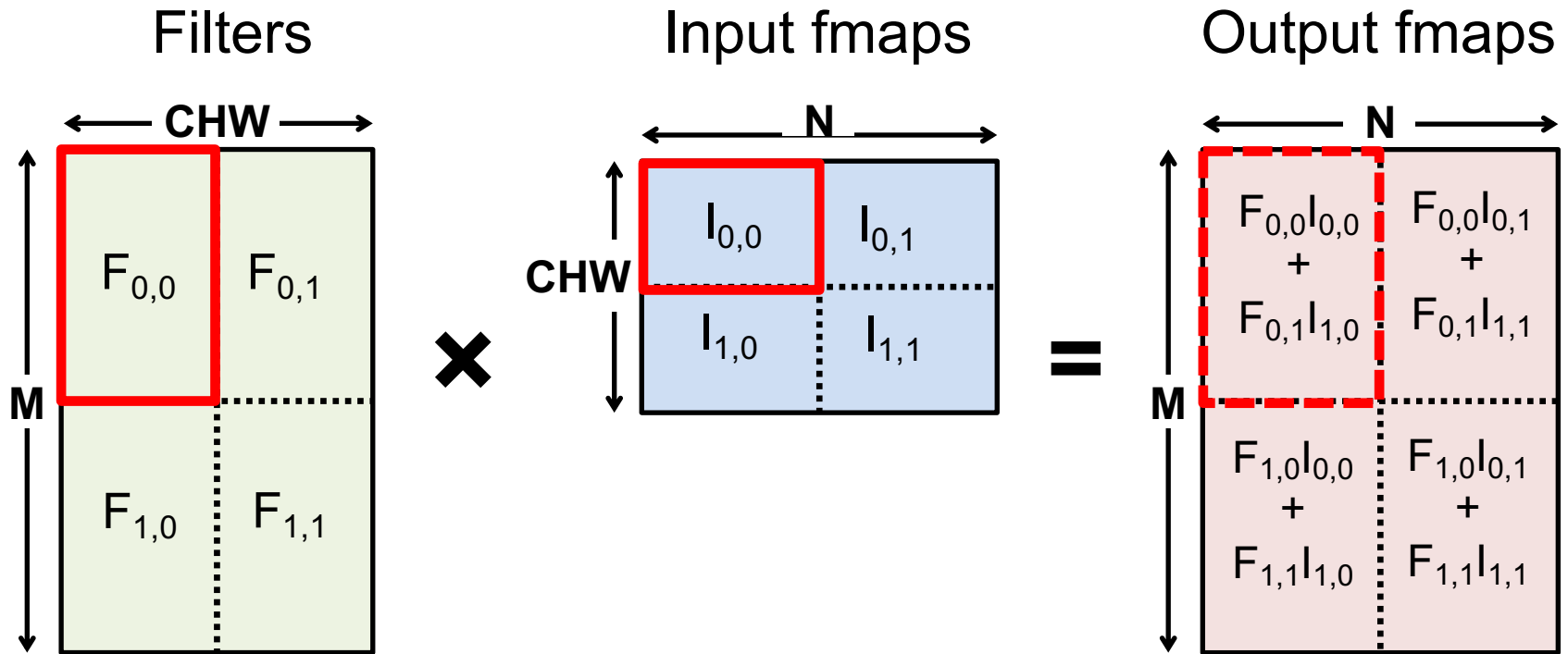
Fully-Connected (FC) Layer



- After flattening, having a batch size of N turns the matrix-vector operation into a matrix-matrix multiply

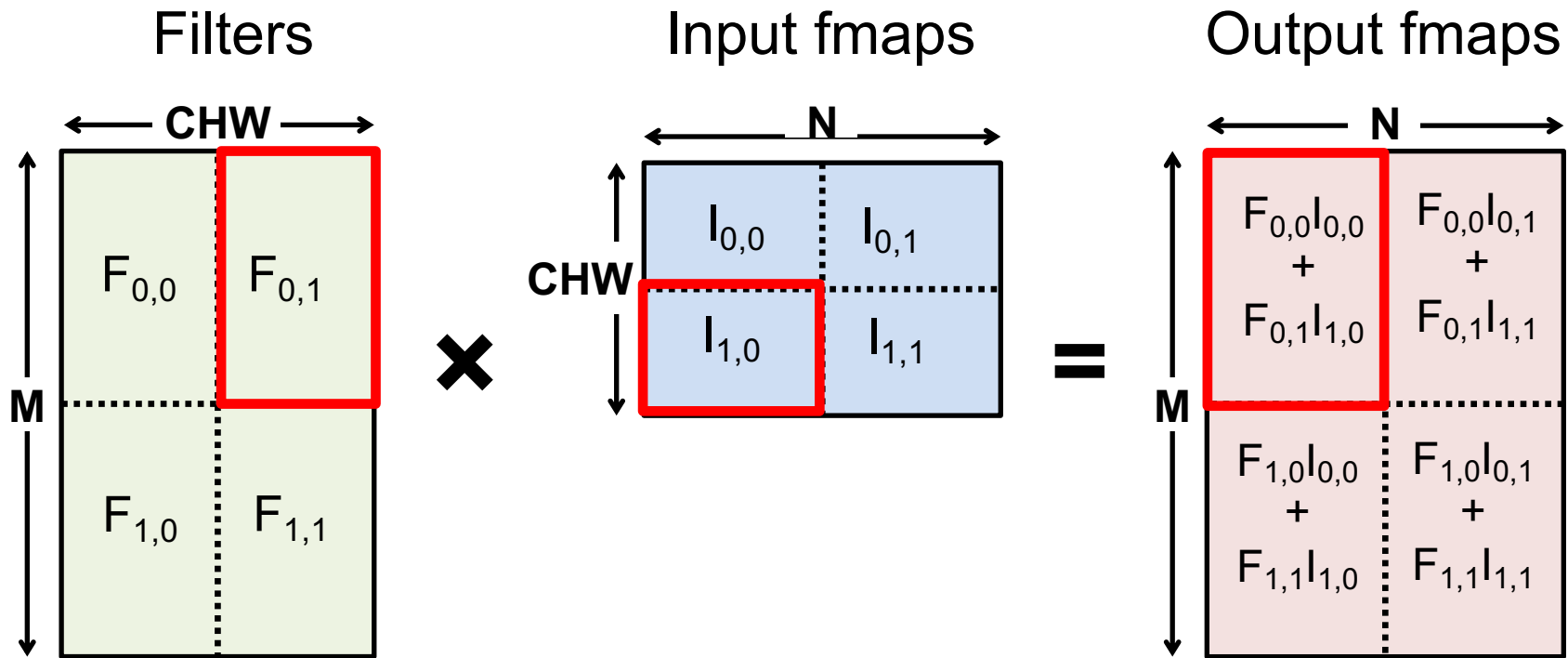
How much temporal locality for naïve implementation? **None**

Tiled Fully-Connected (FC) Layer



Matrix multiply tiled to fit in cache
and computation ordered to maximize reuse of data in cache

Tiled Fully-Connected (FC) Layer



Matrix multiply tiled to fit in cache
and computation ordered to maximize reuse of data in cache

Fully-Connected (FC) Layer

- Implementation: **Matrix Multiplication (GEMM)**
 - **CPU:** OpenBLAS, Intel MKL, etc
 - **GPU:** cuBLAS, cuDNN, etc
- Library will note shape of the matrix multiply and select implementation optimized for that shape.
- Optimization usually involves proper tiling to storage hierarchy

GV100 – “Tensor Core”

$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 FP16 or FP32

New opcodes – Matrix Multiply Accumulate (HMMA)

FP16 operands?

Inputs 48 / Outputs 16

Multiplies?

64

Adds?

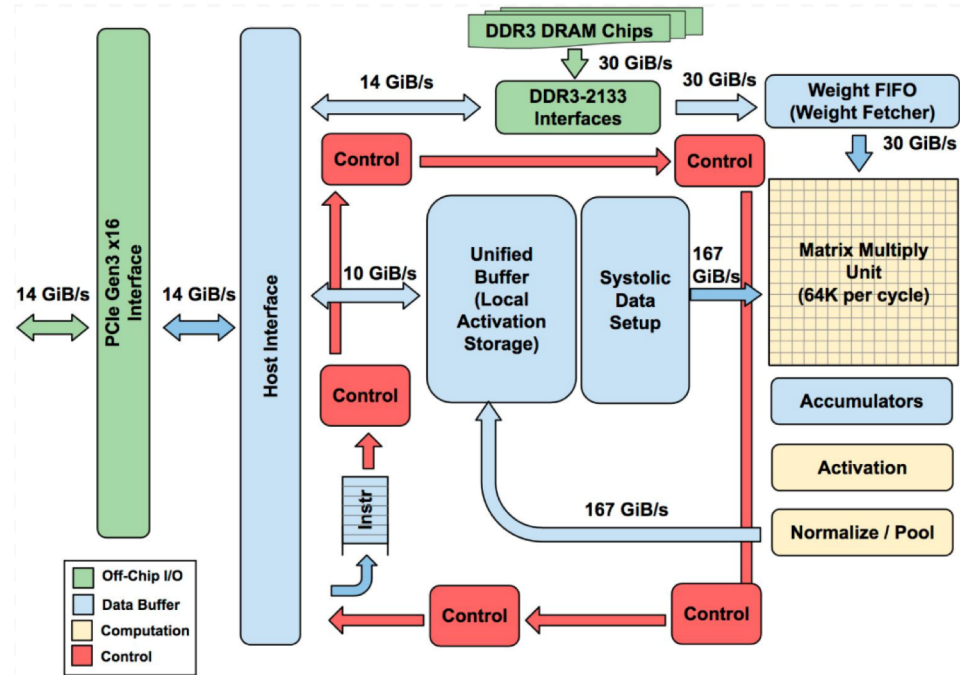
64

Tensor Core....

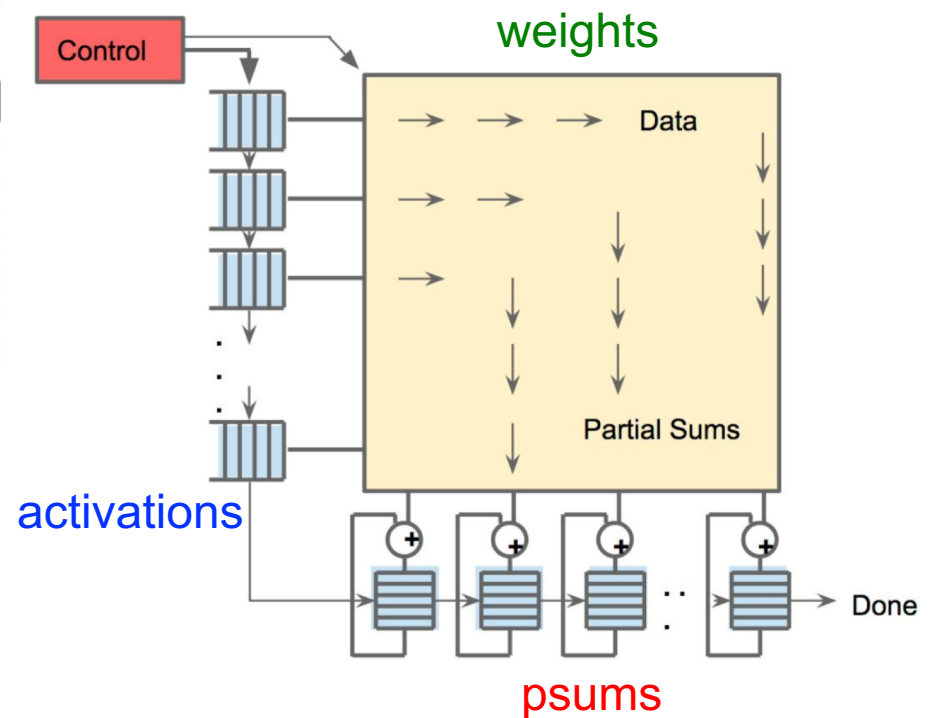
- 120 TFLOPS (FP16)
- 400 GFLOPS/W (FP16)

TPU

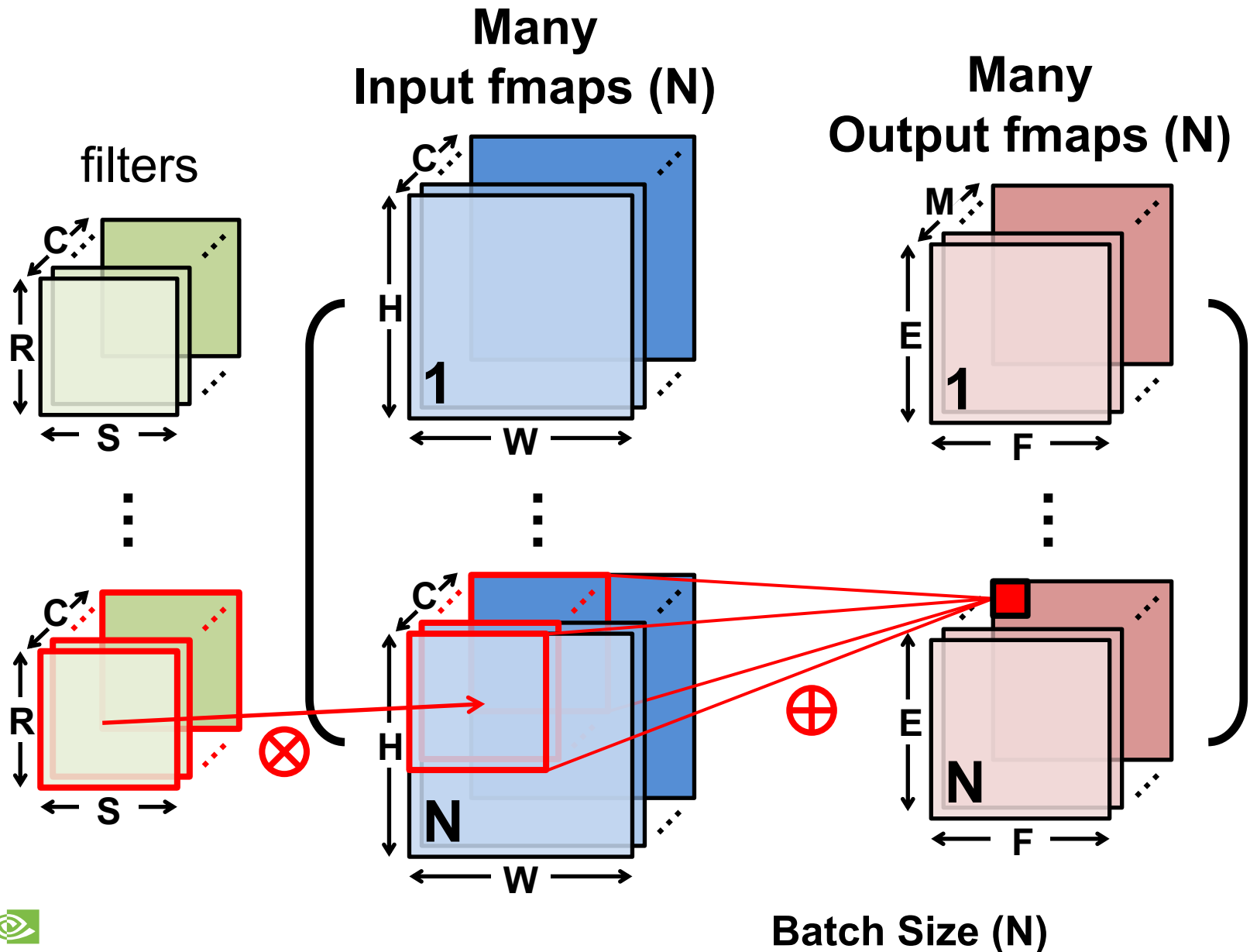
Top-Level Architecture



Matrix Multiply Unit



Convolution (CONV) Layer

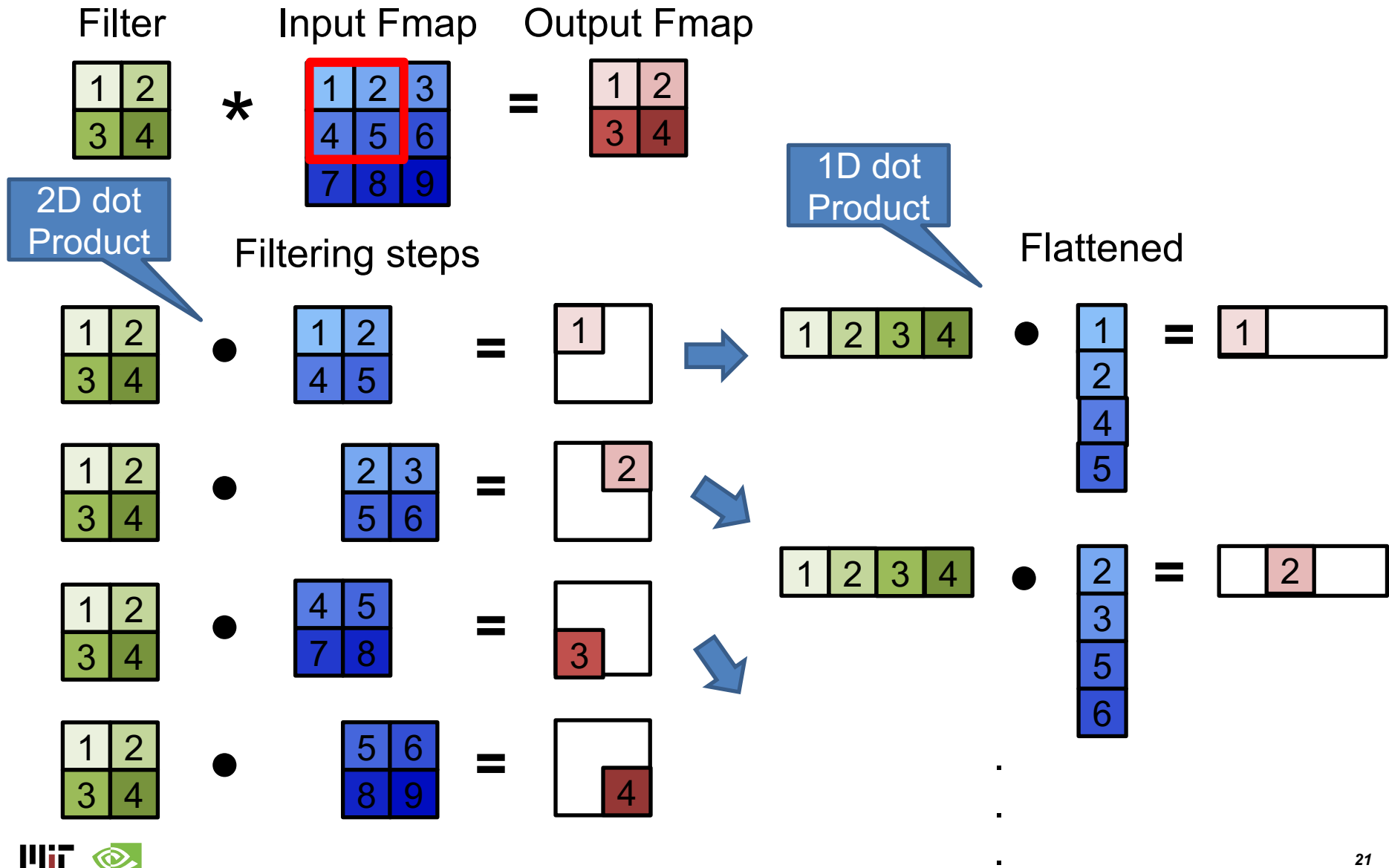


CONV Layer Implementation

Naïve 7-layer for-loop implementation:

```
for (n=0; n<N; n++) {  
  for (m=0; m<M; m++) {  
    for (x=0; x<F; x++) {  
      for (y=0; y<E; y++) {  
        } } } } } for each output fmap value  
  
convolve a window and apply activation {  
  O[n][m][x][y] = B[m];  
  for (i=0; i<R; i++) {  
    for (j=0; j<S; j++) {  
      for (k=0; k<C; k++) {  
        O[n][m][x][y] += I[n][k][Ux+i][Uy+j] × W[m][k][i][j];  
      }  
    }  
  }  
  O[n][m][x][y] = Activation(O[n][m][x][y]);  
}
```

Convolution (CONV) Layer



Convolution (CONV) Layer

Filter Input Fmap Output Fmap

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Convolution:



Flattened

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \bullet \begin{bmatrix} 1 \\ 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} 1 & \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \bullet \begin{bmatrix} 2 \\ 3 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} & 2 & \end{bmatrix} \dots$$

Convolution (CONV) Layer

Filter Input Fmap Output Fmap

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Convolution:



Flattened

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \bullet \begin{bmatrix} 1 \\ 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} 1 & \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \bullet \begin{bmatrix} 2 \\ 3 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} & 2 & \end{bmatrix} \dots$$

Convolution (CONV) Layer

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \bullet \begin{bmatrix} 1 \\ 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} 1 & \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \bullet \begin{bmatrix} 2 \\ 3 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} & 2 & \end{bmatrix} \quad \dots$$

Flattened



Matrix Multiply (by Toeplitz Matrix)

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 5 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$$

Convolution (CONV) Layer

Filter Input Fmap Output Fmap

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Convolution:



Matrix Multiply (by Toeplitz Matrix)

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 5 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$$

Convert to matrix multiply using the **Toeplitz Matrix**

Convolution (CONV) Layer

Filter Input Fmap Output Fmap

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Convolution:



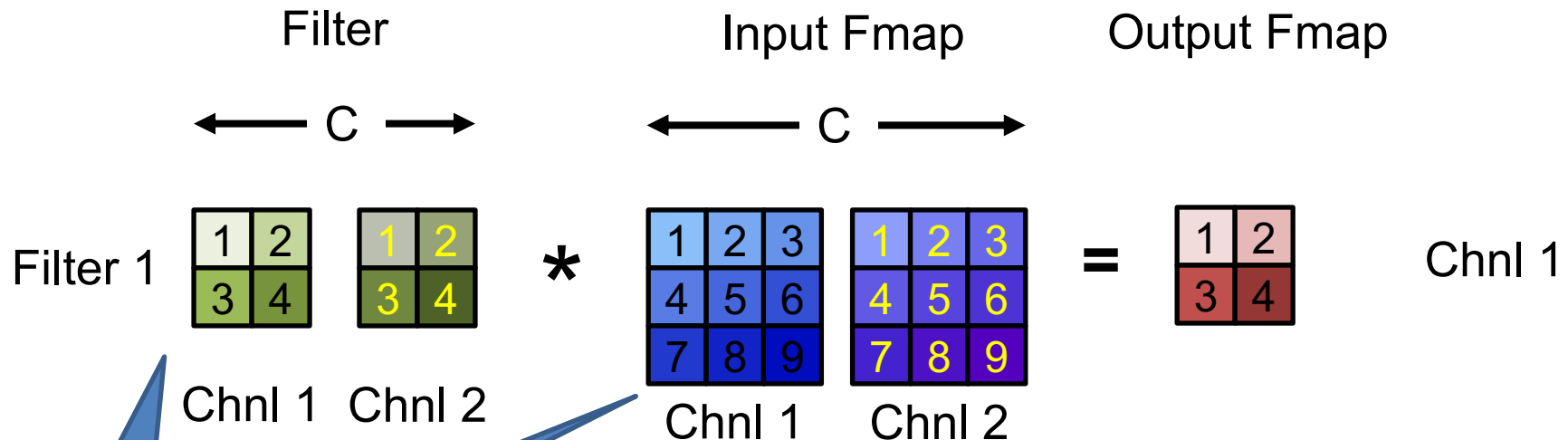
Matrix Multiply (by Toeplitz Matrix)

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 5 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$$

Data is repeated

Convolution (CONV) Layer

- Multiple Input Channels



3D Stacks...

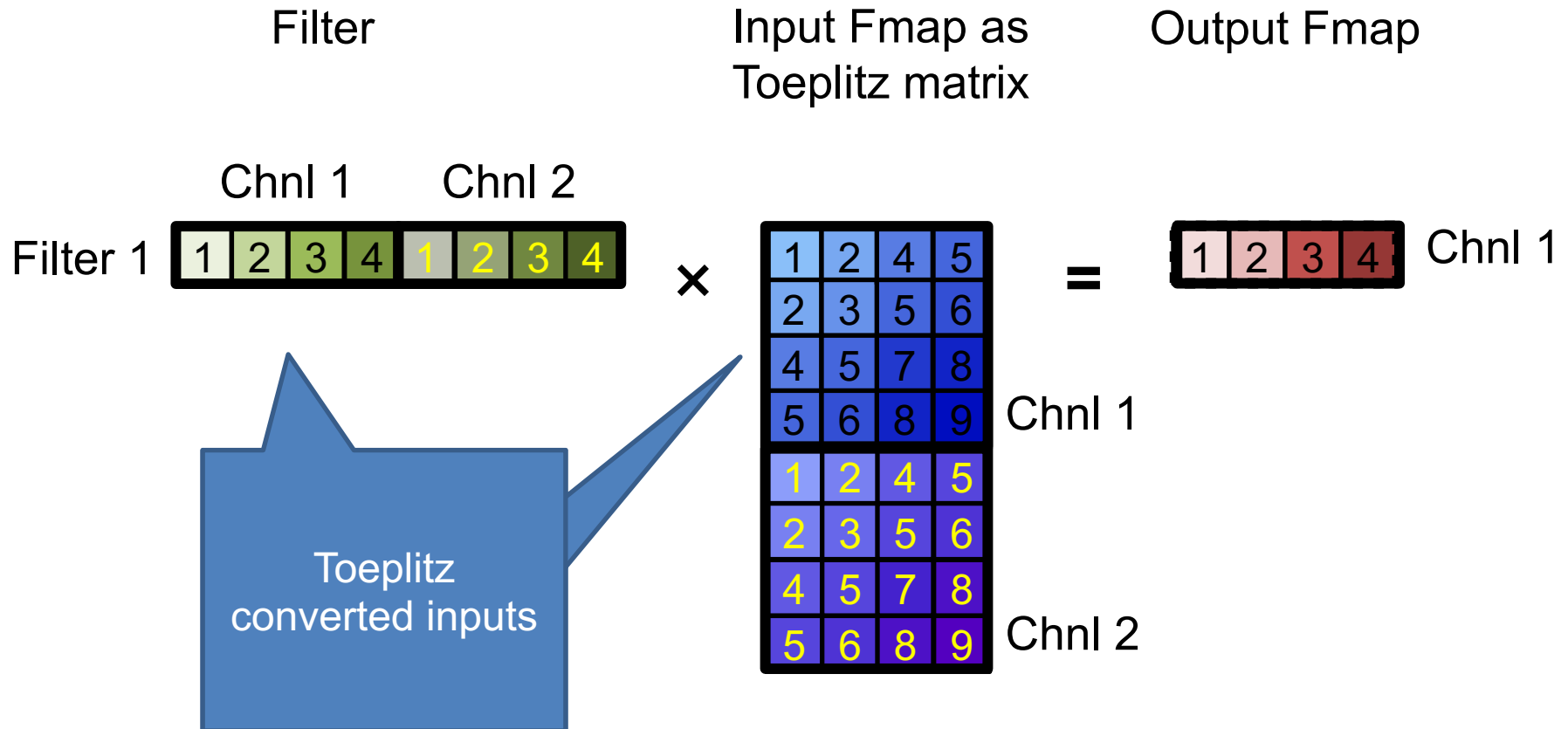
Key:

Black: Input channel 1

Yellow: Input channel 2

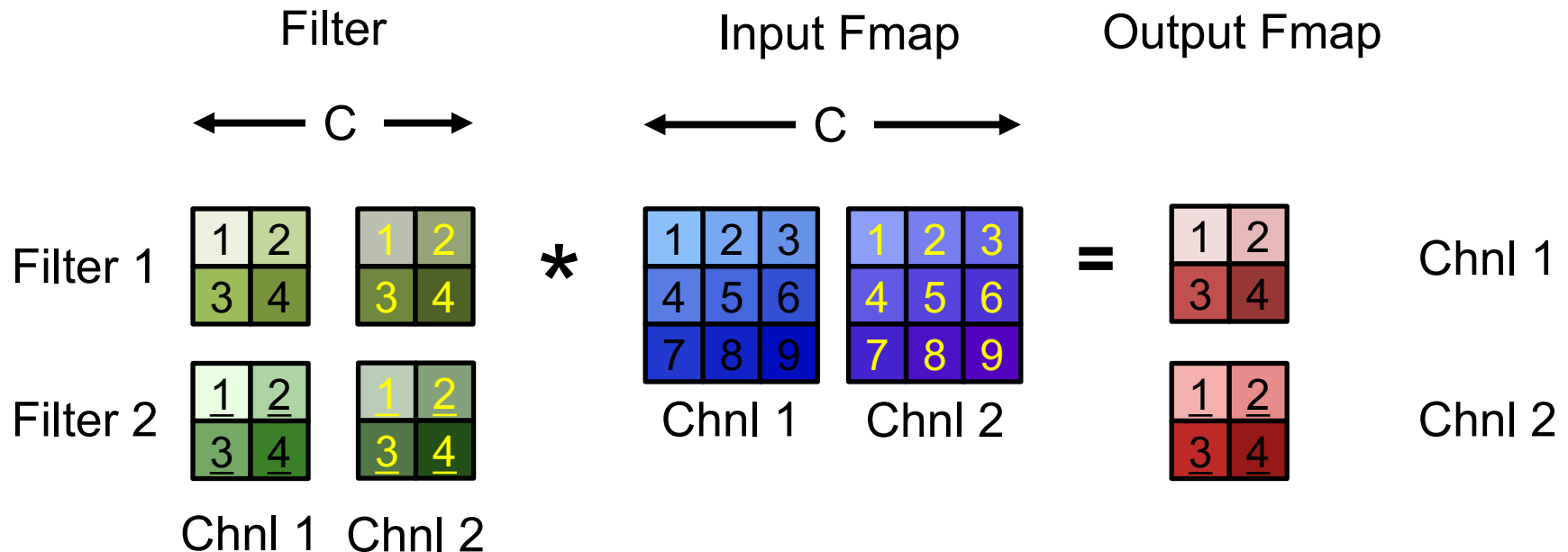
Convolution (CONV) Layer

- Multiple Input Channels



Convolution (CONV) Layer

- Multiple Input Channels and Output Channels



Key:

Black: Input channel 1

Yellow: Input channel 2

Underlined: Output channel 2

Convolution (CONV) Layer

- Multiple Input Channels and Output Channels

Filter

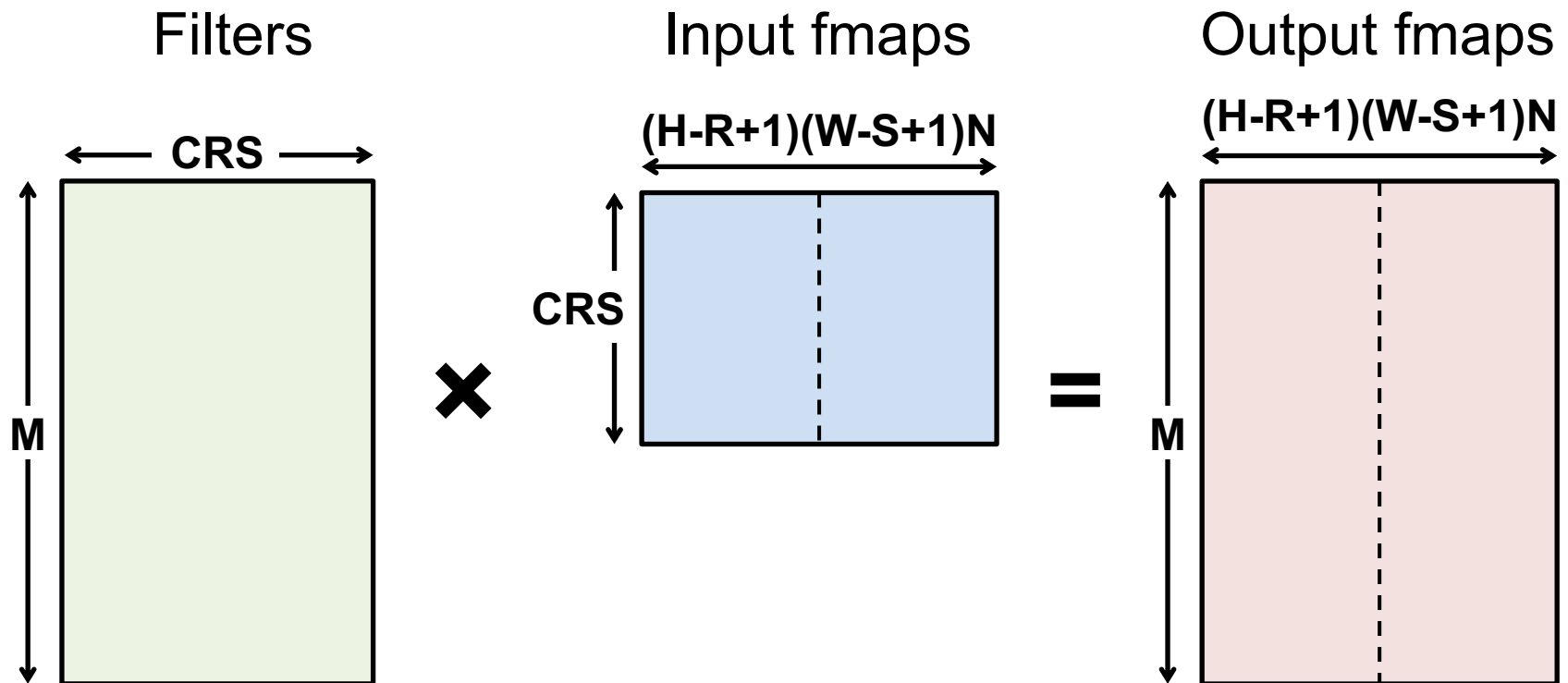
Input Fmap as
Toeplitz matrix

Output Fmap

$$\begin{array}{c}
 \text{Filter} \\
 \begin{array}{cc}
 \text{Chnl 1} & \text{Chnl 2} \\
 \text{Filter 1} & \begin{bmatrix} 1 & 2 & 3 & 4 & 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 & 1 & 2 & 3 & 4 \end{bmatrix} \\
 \text{Filter 2} & \begin{bmatrix} 1 & 2 & 3 & 4 & 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 & 1 & 2 & 3 & 4 \end{bmatrix}
 \end{array}
 \end{array}
 \times
 \begin{array}{c}
 \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 5 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix} \\
 \text{Chnl 1} \\
 \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 5 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix} \\
 \text{Chnl 2}
 \end{array}
 =
 \begin{array}{c}
 \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \end{bmatrix} \\
 \text{Chnl 1} \\
 \text{Chnl 2}
 \end{array}
 \end{array}$$

Convolution (CONV) Layer

- Dimensions of matrices for matrix multiply in convolution layers with batch size N



$N=2$ in example

Computational Transforms

Computation Transformations

- Goal: Bitwise same result, but reduce number of operations
- Focuses mostly on compute

Gauss's Multiplication Algorithm

$$(a + bi)(c + di) = (ac - bd) + (bc + ad)i.$$

4 multiplications + 3 additions

$$k_1 = c \cdot (a + b)$$

$$k_2 = a \cdot (d - c)$$

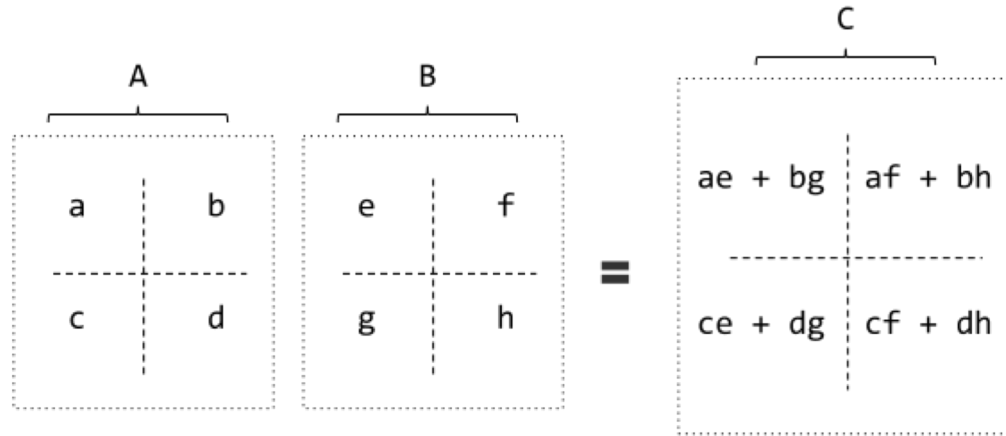
$$k_3 = b \cdot (c + d)$$

$$\text{Real part} = k_1 - k_3$$

$$\text{Imaginary part} = k_1 + k_2.$$

3 multiplications + 5 additions

Strassen



8 multiplications + 4 additions

$$\begin{aligned} P1 &= a(f - h) \\ P2 &= (a + b)h \\ P3 &= (c + d)e \\ P4 &= d(g - e) \end{aligned}$$

$$\begin{aligned} P5 &= (a + d)(e + h) \\ P6 &= (b - d)(g + h) \\ P7 &= (a - c)(e + f) \end{aligned}$$

$$AB = \begin{bmatrix} P5 + P4 - P2 + P6 & P1 + P2 \\ P3 + P4 & P1 + P5 - P3 - P7 \end{bmatrix}$$

7 multiplications + 18 additions

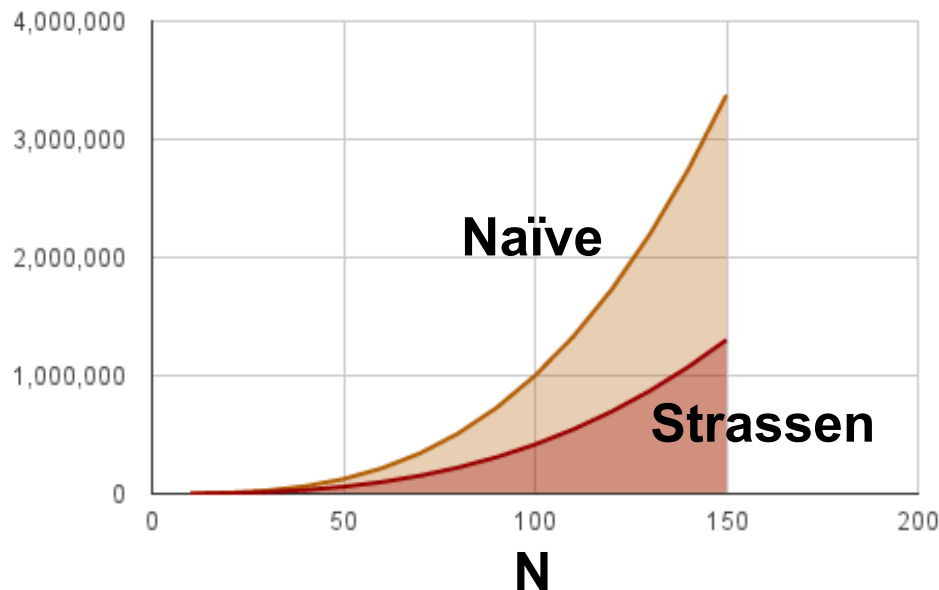
7 multiplications + 13 additions (for constant B matrix – weights)

[Cong et al., ICANN, 2014]

Strassen

- Reduce the complexity of matrix multiplication from $\Theta(N^3)$ to $\Theta(N^{2.807})$ by reducing multiplications

Complexity



Comes at the price of reduced numerical stability and requires significantly more memory

Image Source: <http://www.stoimen.com/blog/2012/11/26/computer-algorithms-strassens-matrix-multiplication/>

Winograd 1D – F(2,3)

- Targeting convolutions instead of matrix multiply
- Notation: F(size of output, filter size)

$$F(2, 3) = \begin{matrix} & \text{input} & & \text{filter} \\ & & & \\ \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} & \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} & = & \begin{bmatrix} y_0 \\ y_1 \end{bmatrix} \end{matrix}$$

6 multiplications + 4 additions

[Lavin et al., CVPR 2016]

Winograd 1D – F(2,3)

- Targeting convolutions instead of matrix multiply
- Notation: F(size of output, filter size)

$$F(2, 3) = \begin{matrix} & \text{input} & & \text{filter} \\ & & & \\ \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} & \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} & = & \begin{bmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{bmatrix} \end{matrix}$$

$$\begin{aligned} m_1 &= (d_0 - d_2)g_0 & m_2 &= (d_1 + d_2)\frac{g_0 + g_1 + g_2}{2} \\ m_4 &= (d_1 - d_3)g_2 & m_3 &= (d_2 - d_1)\frac{g_0 - g_1 + g_2}{2} \end{aligned}$$

4 multiplications + 12 additions + 2 shifts

4 multiplications + 8 additions (for constant weights)

[Lavin et al., CVPR 2016]

Winograd 2D - F(2x2, 3x3)

- 1D Winograd is nested to make 2D Winograd

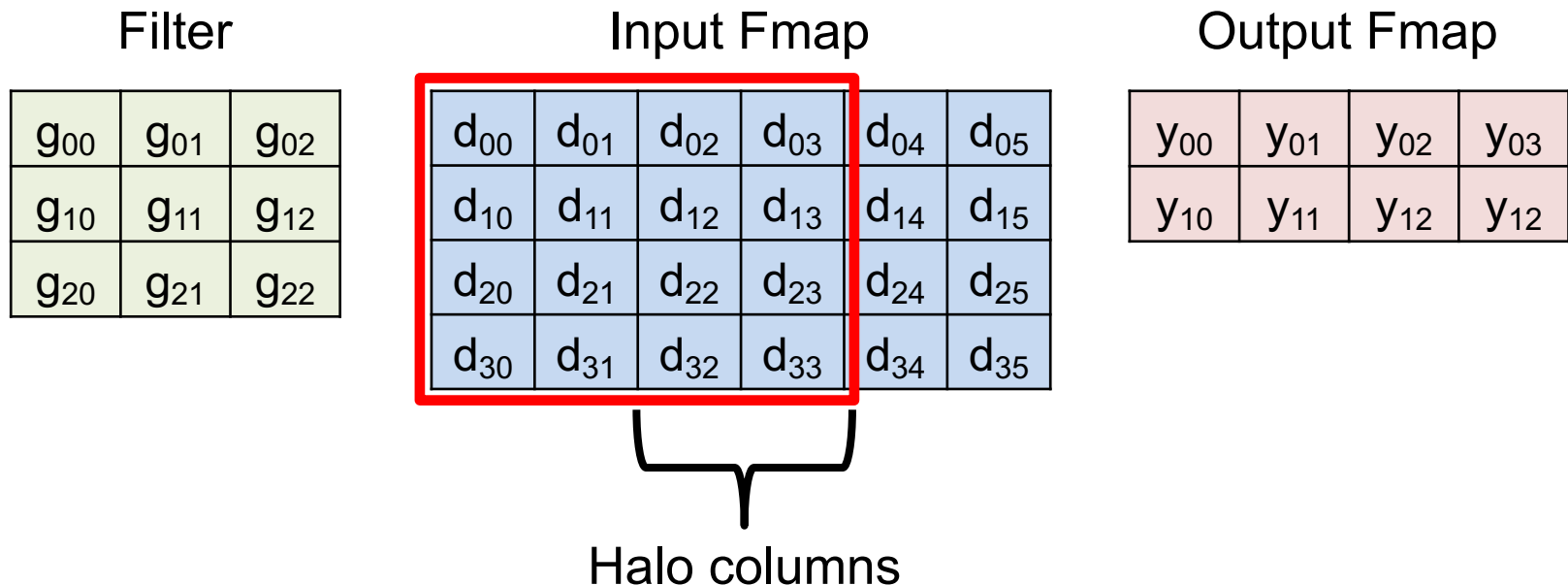
Filter		Input Fmap		Output Fmap																													
<table><tr><td>g_{00}</td><td>g_{01}</td><td>g_{02}</td></tr><tr><td>g_{10}</td><td>g_{11}</td><td>g_{12}</td></tr><tr><td>g_{20}</td><td>g_{21}</td><td>g_{22}</td></tr></table>	g_{00}	g_{01}	g_{02}	g_{10}	g_{11}	g_{12}	g_{20}	g_{21}	g_{22}	*	<table><tr><td>d_{00}</td><td>d_{01}</td><td>d_{02}</td><td>d_{03}</td></tr><tr><td>d_{10}</td><td>d_{11}</td><td>d_{12}</td><td>d_{13}</td></tr><tr><td>d_{20}</td><td>d_{21}</td><td>d_{22}</td><td>d_{23}</td></tr><tr><td>d_{30}</td><td>d_{31}</td><td>d_{32}</td><td>d_{33}</td></tr></table>	d_{00}	d_{01}	d_{02}	d_{03}	d_{10}	d_{11}	d_{12}	d_{13}	d_{20}	d_{21}	d_{22}	d_{23}	d_{30}	d_{31}	d_{32}	d_{33}	=	<table><tr><td>y_{00}</td><td>y_{01}</td></tr><tr><td>y_{10}</td><td>y_{11}</td></tr></table>	y_{00}	y_{01}	y_{10}	y_{11}
g_{00}	g_{01}	g_{02}																															
g_{10}	g_{11}	g_{12}																															
g_{20}	g_{21}	g_{22}																															
d_{00}	d_{01}	d_{02}	d_{03}																														
d_{10}	d_{11}	d_{12}	d_{13}																														
d_{20}	d_{21}	d_{22}	d_{23}																														
d_{30}	d_{31}	d_{32}	d_{33}																														
y_{00}	y_{01}																																
y_{10}	y_{11}																																

Original: 36 multiplications

Winograd: 16 multiplications $\rightarrow$ 2.25 times reduction

Winograd Halos

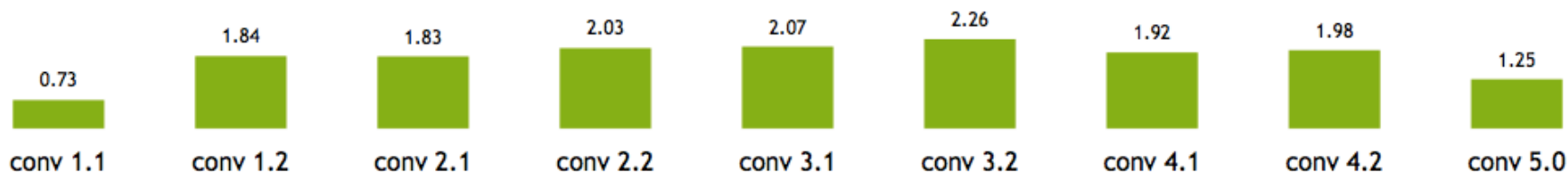
- Winograd works on a small region of output at a time, and therefore uses inputs repeatedly



Winograd Performance Varies

Optimal convolution algorithm depends on convolution layer dimensions

Winograd speedup over GEMM-based convolution (VGG-E layers, N=1)



Meta-parameters (data layouts, texture memory) afford higher performance

Using texture memory for convolutions: **13% inference speedup**

(GoogLeNet, batch size 1)

Winograd Summary

- Winograd is an optimized computation for convolutions
- It can significantly reduce multiplies
 - For example, for 3x3 filter by 2.25X
- But, each filter size (and output size) is a different computation.

Winograd as a Transform

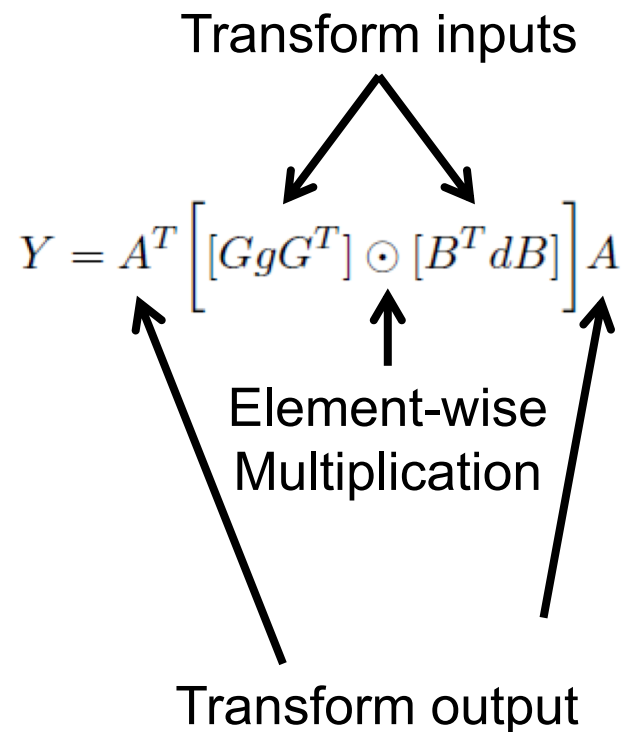
$$B^T = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}$$

$$G = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix}$$

filter $g = [g_0 \ g_1 \ g_2]^T$

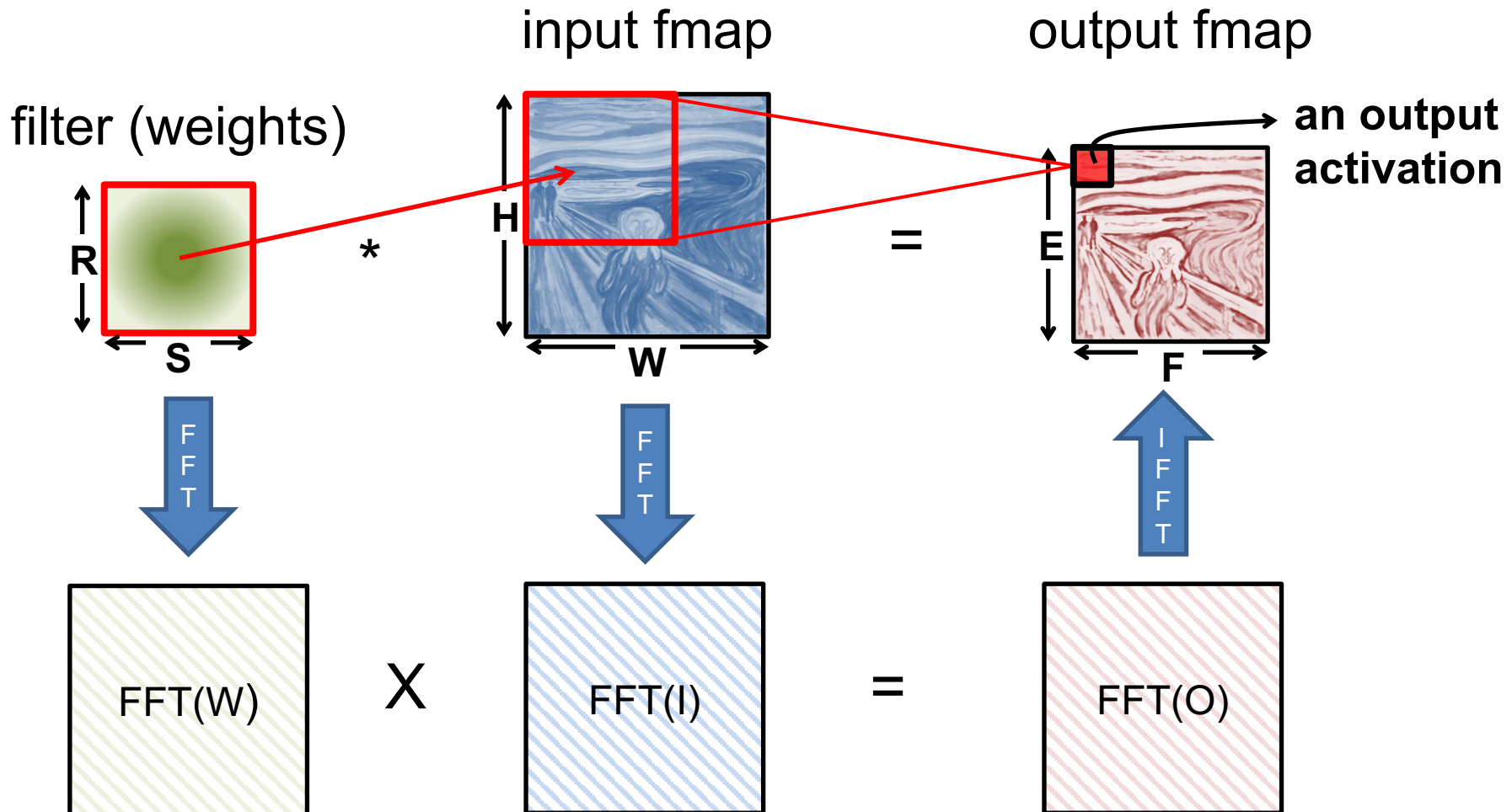
input $d = [d_0 \ d_1 \ d_2 \ d_3]^T$



GgG^T can be precomputed

[Lavin et al., CVPR 2016]

Fast Fourier Transform (FFT) Flow



FFT Overview

- Convert filter and input to frequency domain to make convolution a simple multiply then convert back to space domain.
- Convert direct convolution $O(N_o^2 N_f^2)$ computation to $O(N_o^2 \log_2 N_o)$
- So note that computational benefit of FFT decreases with decreasing size of filter

[Mathieu et al., ArXiv 2013, Vasilache et al., ArXiv 2014]

FFT Costs

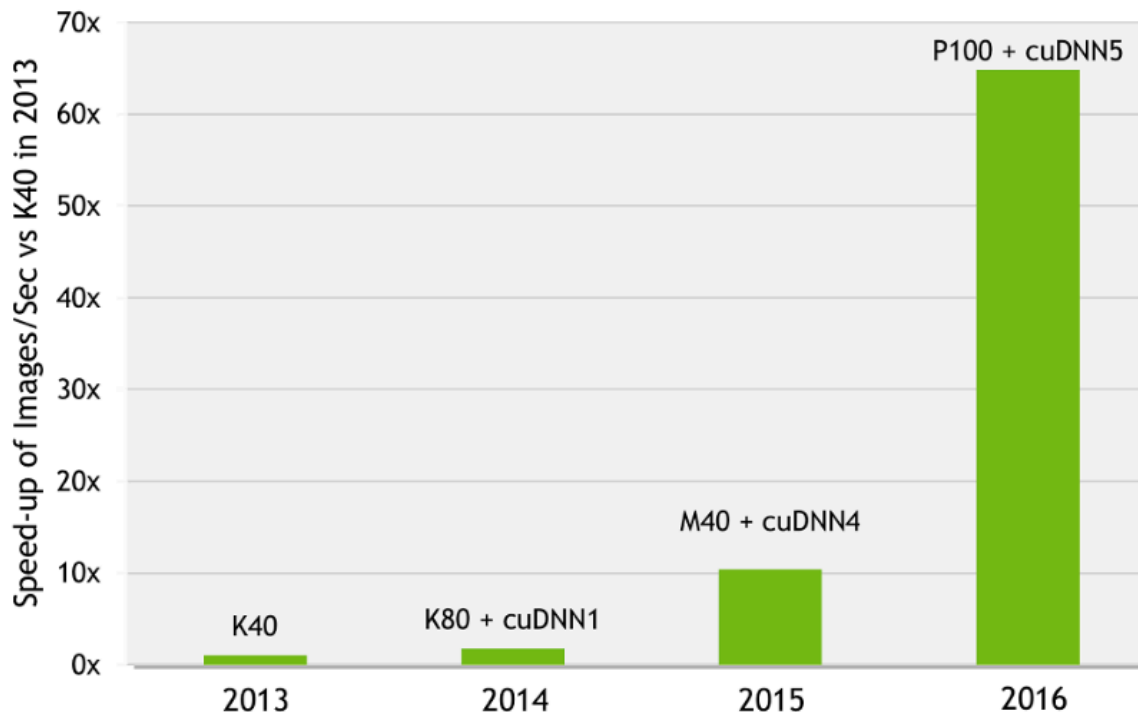
- **Input and Filter matrices are ‘0-completed’,**
 - i.e., expanded to size $E+R-1 \times F+S-1$
- **Frequency domain matrices are same dimensions as input, but complex.**
- **FFT often reduces computation, but requires much more memory space and bandwidth**

Optimization opportunities

- FFT of real matrix is symmetric allowing one to save $\frac{1}{2}$ the computes
- Filters can be pre-computed and stored, but convolutional filter in frequency domain is much larger than in space domain
- Can reuse frequency domain version of input for creating different output channels to avoid FFT re-computations
- Can accumulate across channels before performing inverse transform to reduce number of IFFT

cuDNN: Speed up with Transformations

60x Faster Training in 3 Years



AlexNet training throughput on:

CPU: 1x E5-2680v3 12 Core 2.5GHz. 128GB System Memory, Ubuntu 14.04

M40 bar: 8x M40 GPUs in a node, P100: 8x P100 NVLink-enabled

Source: Nvidia

Optimization Tools

- Halide
- TVM
- Timeloop
-